

Hardware Accelerator for Transformer in Deep Learning

Yi-Chun Hsu

1 Introduction

"Transformer" model is widely used in NLP, but its high computational complexity and intensive operations lead to significant costs on hardware. This work implements hardware accelerator for Transformer, utilizing parallel-processing PE banks to accelerate matrix computations in the attention stage. The design of the multi-port buffer is introduced to overcome the data transmission bottlenecks between different modules. Additionally, piece-wise linear function approximation is employed to the Softmax function, which significantly reduces computational cost while maintaining accuracy.

2 Overall Hardware Architecture

The overall architecture is based on [1] and implemented with reference to [2]. Figure 1 shows the attention stage in Transformer, it can be divided into the following parts:

2.1 PE bank

Referring to the architecture of [3], the large matrix multiplication will be divided into small processing units, and finally a 4×4 kernel. Combined with the PE bank architecture of [4] shown in Figure 2, achieving parallel computation. Each cycle, four values are extracted from the vectors of each input matrices and fed into PE bank.

2.2 Buffer

Since the PE bank enables parallel processing of matrix computations, thus the single-data-per-cycle transmission mode between the buffer and other modules causes data congestion at the I/O, resulting in a bottleneck. To address this issue, a multi-port buffer is employed, allowing multiple data to be transferred at the same time. As the matrix multiplication is divided into 4x4 kernels, the buffer is designed with four read ports and four write ports to support simultaneous read-write operations.

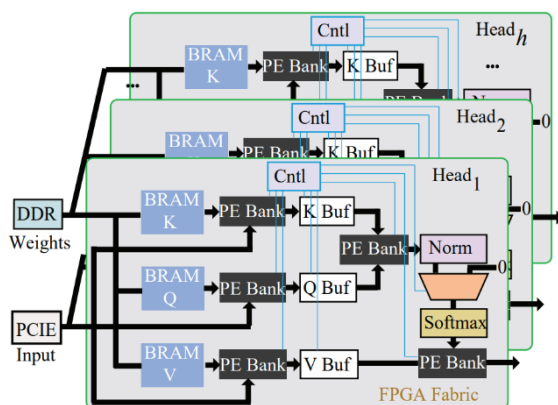


Figure 1: Attention stage

2.3 Softmax

In Softmax, the most time-consuming and resource-intensive operation is the computation of exponential function. Knowing that the input range of Softmax is constrained after normalization, a piece-wise linear function can be used for approximation. Using of LUT may achieve higher accuracy, but it requires a large ROM cost. Therefore, adopting a piece-wise linear function saves significant hardware resources while maintaining good accuracy.

3 PE bank

The input data is set to 12 bits fixed point number by default (4 bits for the integer part and 8 bits for the fractional part). Due to multiplication, the output data expands to 8+8 bits. After normalization, the 16-bit output will be normalized down to 12 bits (4+8) while maintaining the same decimal precision. Each entry in the input matrix is transmitted through the PE bank over 4 clock cycles (Figure 2). Each PE (Processing Element) receives two separate inputs: the output from the PE to its left and above it in the previous clock cycle. Each cycle, after performing the multiplication, the PE passes the received values further along the chain.

Each PE stores the accumulated multiplication, and the final accumulated value is the entry of the output matrix. If the accumulated value exceeds the maximum limit of the available bit range, saturation is applied to cap the output at the maximum value and thus avoid overflow. Due to saturation, the input is practically set to accept 16 bits (8+8) to handle scenarios where the two numbers being multiplied have a significant difference in magnitude.

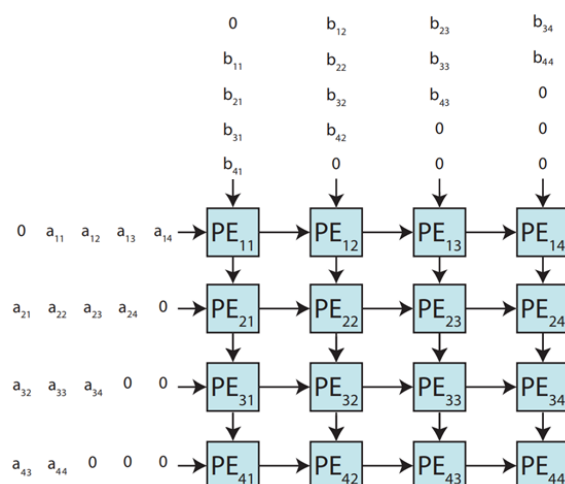


Figure 2: PE bank

4 Softmax

4.1 Preprocessing

For each input data x_i , do the preprocessing as below:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}} = \frac{e^{x_i - \max(x_i)}}{\sum_{j=1}^n e^{x_j - \max(x_i)}} = \frac{e^{y_i}}{\sum_{j=1}^n e^{y_j}}$$

The new data is defined as $y_i = x_i - \max(x_i)$. Thus, all input data are nonpositive, which is better for estimation since the exponential function has a milder slope on the negative side. It's obvious that the Softmax result remains unchanged after preprocessing.

4.2 Data form

Since the data after normalization is generally not big, this work assumes the input data range for Softmax to be $[0, 8]$. After preprocessing, the range becomes $[-8, 0]$. Note that most works set range to $[0, 10]$, but this work sets the upper bound to 8 since 8 is a power of 2. The input data is 12 bits 2's complement number (4+8). The output data of exponential function and Softmax are both 16 bits and all bits are fraction since all outputs are less than 1.

4.3 Exponential function

Referring to [5], a lookup table (LUT) is introduced to estimate exponential function. But LUT requires a large ROM to store the data. Thus, this work applies a piece-wise linear function for approximation and saves cell area. The 7 segments of the piece-wise linear function are shown in Table 1. Also, there must be at least one 0 after the preprocessing, so we can output 1 directly for input 0. Compared to 6-segment version, 7-segment one achieves higher accuracy with only a slight increase in cell area (as shown in Table 2). In Table 2, the mean square error (MSE) is scaled by 2^{32} when comparing the errors because of the 16-bit representation for fraction :

$$\text{MSE}_{\text{new}} = \frac{(\text{error} \times 2^{16})^2}{\text{Number of Data}}$$

The final Softmax result will be presented later to demonstrate that the piece-wise linear function method maintains high accuracy.

Input Range	Corresponding Linear Function
$-8 \leq x \leq -6$	$y = 0.00106811x + 0.00854492$
$-6 < x \leq -4$	$y = 0.00833123x + 0.04995728$
$-4 < x \leq -2$	$y = 0.05860901x + 0.23432922$
$-2 < x \leq -1$	$y = 0.02347717x + 0.58473206$
$-1 < x \leq -0.5$	$y = 0.48832703x + 0.84350586$
$-0.5 < x \leq -0.25$	$y = 0.69598389x + 0.95008850$
$-0.25 < x \leq 0$	$y = 0.89477539x + 0.99650573$

Table 1: Linear functions

Type	$\text{MSE} \times 2^{32}$	Cell Area(μm^2)
6 segments v1	178433	17702
6 segments v2	169954	15821
7 segments v1	123924	19158

Table 2: Comparison of different segments

5 Result

The synthesis is performed using Synopsys Design Compiler (DC) in a TSMC 0.13 μm process.

For matrix multiplications, each processing element (PE) takes 20081 μm^2 area. However, by top-down compilation, the overall cell area for PE bank (each 16 PEs) can be optimized to 250746 μm^2 . The critical path delay of PE bank is 7.21 ns, which is consistent to single PE due to the pipelined design. Actually, the cell area of PE bank can be further reduced by applying looser timing constraints since the PE bank is not the critical path of the overall Transformer design.

The exponential module requires 19158 μm^2 area, and the overall Softmax requires 117045 μm^2 , with the divider accounts for the majority of area. The critical path delay of Softmax is 17.9 ns due to 32 bits / 20 bits division, which is also the critical path of whole module.

The estimation result is shown in Table 3. Since we concern about large data (probabilities) in Softmax, the error calculation is performed as below:

$$\text{Average Error} = \frac{\sum |\text{real value} - \text{output}|}{\text{largest real value} \times \text{Number of data}}$$

Note that for the cases where a single input dominates (Set 10), the accuracy is pretty high. For the cases where all input data are relatively evenly distributed across the input range, the errors are slightly larger. However, in all cases, the errors are less than 1 %, demonstrating the high accuracy of the estimation method.

Test Set	Pattern	Error
Set 1	0 -0.75 -1.0 -1.25 -1.5 -1.75 -2.0 -2.25 -2.5 -2.75 -3.0 -3.25 -3.5 -3.65 -4.0 -4.25	0.60%
Set 2	0 -0.7 -0.8 -1.0 -2.5 -2.7 -2.6 -3.0 -3.1 -3.2 -3.3 -3.4 -3.5 -3.7 -3.4 -4.2	0.57%
Set 3	0 -0.3 -1.2 -1.5 -1.7 -2.2 -2.5 -2.6 -2.7 -3.0 -3.2 -3.5 -3.7 -4.0 -4.2 -5	0.51%
Set 4	0 -0.6 -0.7 -0.8 -0.9 -1.2 -1.4 -1.7 -3.8 -4.5 -4.6 -4.7 -5.0 -6.5 -6.6 -6.9	0.26%
Set 5	0 -2.3 -2.46 -2.48 -3.14 -3.25 -3.26 -3.64 -3.79 -3.88 -4.02 -4.05 -4.3 -5.6 -5.7 -6.6	0.42%
Set 6	0 -0.51 -2.54 -2.56 -2.78 -2.8 -2.89 -3.45 -3.67 -3.58 -3.68 -3.96 -3.99 -4.25 -5.6 -6.3	0.66%
Set 7	0 -0.4 -0.56 -0.78 -0.98 -1.25 -1.5 -1.68 -1.9 -2.6 -2.99 -3.58 -3.68 -3.78 -3.98 -4.68	0.71%
Set 8	0 -1.25 -1.88 -2.58 -3.6 -3.68 -3.69 -3.77 -3.78 -3.79 -3.99 -4.6 -4.8 -5.2 -5.24 -5.36	0.84%
Set 9	0 -0.42 -0.46 -0.48 -0.486 -0.49 -0.51 -0.56 -0.58 -0.589 -0.594 -0.61 -0.62 -0.65 -0.75 -0.81	0.25%
Set 10	0 -5.13 -5.22 -5.41 -5.54 -6.01 -6.03 -6.12 -6.14 -6.18 -6.54 -6.58 -6.89 -7.03 -7.06 -7.12	0.04%

Table 3: Estimation result of Softmax

6 References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, 2017.
- [2] B. Li, S. Pandey, H. Fang, Y. Lyv, J. Li, J. Chen, M. Xie, L. Wan, H. Liu, C. Ding, FTRANS: Energy-

Efficient Acceleration of Transformers using FPGA, 2020.

- [3] S. Markidis, S. W. D. Chien, E. Laure, *KTH royal Institute of Technology*, I. B. Peng, J. S Vetter, *Oak Ridge National Laboratory*. NVIDIA Tensor Core Programmability, Performance & Precision, 2018.
- [4] Shaarada D. Yamini, Ganesh S. Mirishkar, Anil Kumar Vuppala, Suresh Purini. Hardware Accelerator for Transformer based End-to-End Automatic Speech Recognition System, 2023.
- [5] Xiao Dong, Xiaolei Zhu, De Ma. Hardware Implementation of Softmax Function Based on Piecewise LUT, 2019.